

1 Requirements

In this section, I just want to explore what the primitives need to be able to do so that I approach their implementation in a structured way.

Ultimately, the primitive transformations modify the world in a specific way. They apply a transformation to the world state. In some cases that is a new participant and in other cases, it is an existing participant that is modified. The point here is that the transformations are meaningful and they exist as part of a narrative.

1.1 Add

So lets start with the basic add primitive:

This transformation needs two participants, `addend1` and `addend2`. Both `addend1` and `addend2` must be numbers, they can be whole numbers, or floats. The resulting participant is a float or an int. So the meaning of the participants correctness is made even more unambiguous and the invariants can be used to predict when this transformation will fail. This allows the engine to reason about the narrative leading up to this transformation and eliminate semantically invalid states.

So it is clear that I also need meaningful primitive types for the participants. The meaning of `addend1` and `addend2` are that they are part of a transformation but also must be of type float or int. So I need to establish that meaning and there is a mechanical sequence realizes the meaning. If I check if the participants type has the correct meaning, it will invoke the mechanical implementation to access the type.

So the transformation accepts two participants and generates a participant that is the result of the transformation. The types of all the participants are known and the invariant states that both `addend1` and `addend2` have to be of type float or int. This is the meaning of the transformation and the conditions under which it will succeed.

Using this, the transformation can be invoked by a narrative. So I need to also provide the syntax for how it can be invoked by a narrative. For example, store result of add `addend1` and `addend2` in `result_name`. The narrative can be parsed and the actual participants can be identified using this.

So now I also set up parsing rules for the transformation. If I see the keyword `add`, then I know how to parse the rest of the line, so I can use the parser to access the structure from the primitive's meaning.

1.2 General Rule

In general, this process is assigning meaning to the primitives of a programming language and transforming programming into an act of constructing meaning. It is creating a semantic toolkit that consists of the meaningful primitives and using those primitives to construct more meaning. Since the primitives are computable, the composed meaning remains computable.

Ultimately, this is the meaning behind the mechanics. So even the type of the participant has a specific meaning, float, int, boolean etc. So if the meaning of `add` is that it must have addends that are a float or an int, then the correctness can be reasoned about by the engine and semantically invalid states can be eliminated.

Its actually very simple, this is just programming meaning. Its just hard to rewire the mind to think of programming this way because it is not doing anything new but it is just unambiguously constructing the meaning.

I actually think the way to become comfortable with this is to think about this as world building. I only say this because the word programming in the traditional sense has a lot of baggage, it evokes images of code sprawling across a screen. Now it is more accurate to think of it as building a world that has meaning and narratives that reason about that meaning. The engine is another form of automated reasoning about that meaning. Through this process, programming is elevated of building deterministic intelligence.

2 Addition

2.1 Multiplication

In this section, I want to explore how the add primitive can be used to build the meaning of multiplication. Multiplication is just repeated addition, so multiplication it is a great case study for how to build more meaning using narratives built from primitives.

However, clearly I can establish multiplication on it own as a primitive because its essentially a primitive operator and there are clearly many optimizations, it is just a very simple case study that shows how meaning can be built on top of primitives.

So despite me establishing the meaning of multiply as a primitive, if I were to proceed with designing multiplication using repeated addition, I would do it as follows:

Multiply multiplicand and multiplier.

Store 0 in result. Store 0 in counter. While the counter is less than multiplier, store result of add multiplicand and result in result then store result of add 1 and counter in counter.

It is clear from this that to create the design that realizes this narrative, you need to establish how while will be automatically translated into a design structure. In this case, it is simply a conditional check that counter is less than the multiplier and apply the subsequent store operations. This can be automatically constructed as a design from the narrative and then an engine can reason about it.

Note that I don't specify the invariants in the narrative, since the transformations are built from computable semantic primitives, the invariants are established through construction.

Note that the conditional check in the while loop isn't a mechanical, it is inherently meaningful. It is reasoning about the state of the world and making a decision about which behavior to exhibit. If it fails, the meaning of that failure is unambiguous.

2.2 Bill Total Example

Using the add primitive, you can establish the meaning of behaviors. For example consider the behavior below in a restaurant payment app which is called add bill.

Store 0 in bill-total. For each diner in the group, store the result of add diner-owes and bill-total in bill-total.

Now in a larger narrative you can use add bill

After the diners request the bill, store the result of add bill in total-owed.

Since add bill is built from primitives that have unambiguous invariants, the correctness of the narrative can be established. For example if the each diners owes's type is not a number, the engine can reason that the add will fail.

3 Composing Meaning

In the previous section I established a narrative using the add primitive to a bill total example. In this section, I will continue exploring how the primitives can be used compose more meaning and how that meaning can be used to compose more meaning. I am not so focused on being exact with how the narrative will be parsed, I am more interested in exploring how meaning can be composed. I will become more precise with the parsing as I continue with the primitives.

Lets consider a bubble sort algorithm. As a quick reminder, a bubble sort algorithm can be generalized as shown below. The initial participant is an unsorted list and I will list the remaining participants after sharing the sorting steps below.

1. Initialize right end to size of list.
2. Set current position to first element of list.
3. Compare current element to adjacent element to its right.
4. Swap if not in order.
5. Move onto next pair.
6. Repeat steps 3,4 and 5 until right end is reached.
7. Decrease right end by one to indicate sorted items.
8. Repeat steps 2-7 until entire list is sorted.

Now we can establish the design of a bubble sort algorithm and establish the narratives that exist in it. Then I will compose the meaning of bubble sort by establishing the meaning of each part of its narrative.

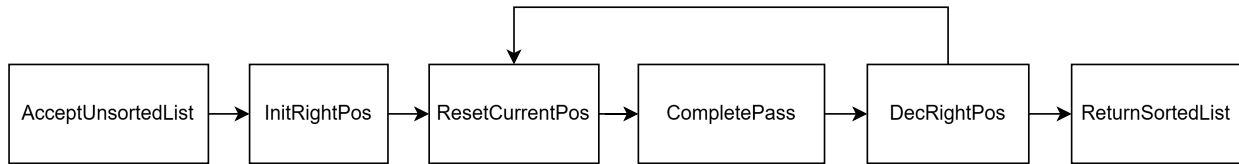


Figure 1: Bubble Sort Behaviors and Narratives

The narratives of the design can be described as follows:

Accept an unsorted list and initialize the right position.
Complete pass then decrement right position. If right position is 0, then return sorted list
If it is greater than 0, then reset current position and complete another pass.

At this level, we have the following participants:

1. Current Position: Current position being inspected.
2. Right Position: Elements to the right of this position are sorted.
3. Unsorted List: List of numbers to be sorted

3.1 AcceptUnsortedList

3.2 InitRightPos

Store the length of the list in size
then
Set rightPosition to size

In this case, there are two primitive transformations and added "then" between the transformations to indicate that they happen in order.

So I first get the length of the list and then I store the result in the size participant. This is a primitive because I can establish it using the built in operator. If I wanted to implement it myself, I can implement the meaning of length by building a design that will count the number of elements.

Next, I set rightPosition to the size of the list. Both of these simply use the assign operator and it is a primitive. Now the meaning of InitializeRightPosition has been constructed from the primitives.

3.3 ResetCurrentPosition

Set leftPosition to 0

The meaning of this behavior is trivial, it simply sets the participant leftPosition to 0.

3.4 Complete Pass

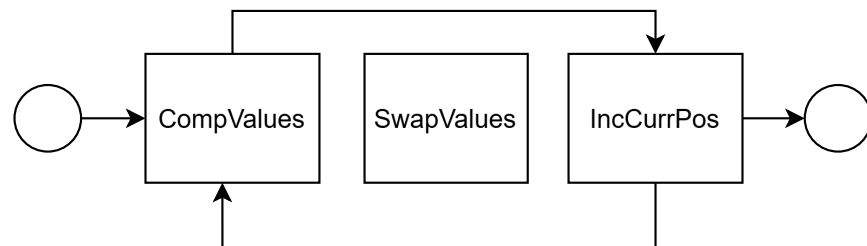


Figure 2: Complete Bubble Sort Pass

3.4.1 CompValues

Set leftPosition to currentPosition
 then
 Store result of add 1 and leftPosition in rightPosition
 then
 Set leftValue to unsortedList value at leftPosition
 then
 Set rightValue to unsortedList value at rightPosition.
 then
 Store result of left position greaterThan rightPosition in shouldSwap

In this case, the meaning of the compareValues behavior is built from multiple primitive transformations. First, there is the direct assignment of one value to another participant. Then an add primitive is executed and the result is saved in a participant. Then the value at the left and right position is saved in the relevant participants. Finally a comparison is done to set the value of shouldSwap participant.

So here, we can see two interesting invariants that can be reasoned about. The left and right position are being accessed from the unsortedList and there is an invariant for those primitives that they must be within the size of the list.